

Food Spoilage Detection System

An IoT and Machine-Learning Approach to Real-Time Food Freshness Monitoring

Tanzim Ahamed | Umme Kulsum | Eiftyak Moon

Department of Information and Communication Engineering (ICE), Daffodil International University

August 10, 2026

Abstract

Food spoilage is typically detected only after visible or olfactory signs appear, by which point contamination is already advanced. This project presents a full-stack Internet of Things (IoT) system that continuously monitors gas concentration, pH, temperature, and humidity around stored food using an ESP32 microcontroller, and classifies freshness in real time as Fresh, Warning, or Spoiled. Readings are transmitted to a PHP/MySQL backend and visualized on an auto-refreshing web dashboard, while local LED and buzzer indicators provide feedback even without network access. A threshold-based rule provides an immediate baseline classification, and an optional Random Forest classifier, trained with Scikit-learn on accumulated sensor readings, offers a data-driven upgrade path as more labelled data is collected. The system was validated end-to-end using both simulated HTTP requests and live sensor data, confirming consistent status classification across the firmware, API, and machine-learning layers. The result is a low-cost, extensible monitoring pipeline suitable for household or small-scale storage environments.

Keywords: *Internet of Things, food spoilage detection, ESP32, embedded sensors, machine learning, real-time monitoring*

1 Introduction

Stored food quality is conventionally assessed by manual inspection, which is subjective, infrequent, and reactive rather than preventive. This project addresses that gap with an automated pipeline that senses key spoilage indicators continuously and reports status changes as they happen, rather than after spoilage is already visible.

The objectives of the system are to: (i) continuously monitor gas, pH, temperature, and humidity without manual inspection; (ii) provide immediate on-device alerts through LEDs and a buzzer, independent of network availability; (iii) persist every reading for historical analysis; (iv) present live and historical trends through a web dashboard; and (v) enable a transition from a fixed threshold rule to a trained classifier as data accumulates.

The system targets a single monitoring node running on a local network via Laragon (Apache, MySQL, PHP), though the underlying schema and API are designed to extend to multiple nodes.

2 Materials and Methods

2.1 System Architecture

The pipeline is organized into four layers. The sensing layer (ESP32 with an MQ-135 gas sensor, a pH sensor, and a DHT11 temperature/humidity sensor) reads raw values, computes a local status, drives an LCD and

status indicators, and posts each reading over Wi-Fi. The API layer (PHP scripts) receives, validates, and stores readings, and serves both live and historical data as JSON. The storage layer is a MySQL database with sensor_data and alerts tables. The presentation layer is a browser dashboard that polls the API every five seconds and renders live tiles and Chart.js line charts. An optional intelligence layer, written in Python, trains and applies a Random Forest classifier.

2.2 Hardware and Wiring

The MQ-135 and pH sensor analog outputs connect to ADC1 pins GPIO34 and GPIO35 respectively; the DHT11 data line connects to GPIO4; a 1602 LCD communicates over I²C (SDA/GPIO21, SCL/GPIO22); a buzzer is driven from GPIO25; and green, yellow, and red status LEDs are driven from GPIO26, GPIO27, and GPIO14 through 220 Ω resistors. The pH sensor's raw ADC-to-pH conversion uses a linear approximation that should be recalibrated against pH 4, 7, and 10 buffer solutions for quantitative accuracy.

2.3 Status Classification Rule

A threshold rule is implemented identically in the firmware, the insertion API, and the prediction fallback, so classification is consistent regardless of connectivity: a reading is Spoiled when gas exceeds 400 and pH is below 5.5; Warning when gas exceeds 250; and Fresh otherwise. This rule also determines the local LED/buzzer response on the ESP32, so alerting continues even during a network outage.

2.4 Database and API Design

The sensor_data table stores gas, pH, temperature, humidity, computed status, and a timestamp for every reading; the alerts table logs a message whenever a reading is not Fresh, referencing the originating row. The insert.php endpoint accepts form or JSON POST bodies, validates that all four fields are numeric, computes status, and writes both tables as needed. The get_data.php endpoint serves the latest reading, a bounded history window, or recent alerts, selected via a type query parameter. All queries use PDO prepared statements.

2.5 Machine Learning Component

train_model.py loads all historical readings from MySQL, performs an 80/20 stratified train-test split, and fits a Random Forest classifier (200 estimators) on the four sensor features to predict status. predict.py applies the trained model when available and otherwise falls back to the threshold rule, ensuring the command-line prediction tool functions before sufficient training data exists. A minimum of roughly 200 labelled readings spanning all three classes is recommended before training is expected to generalize well.

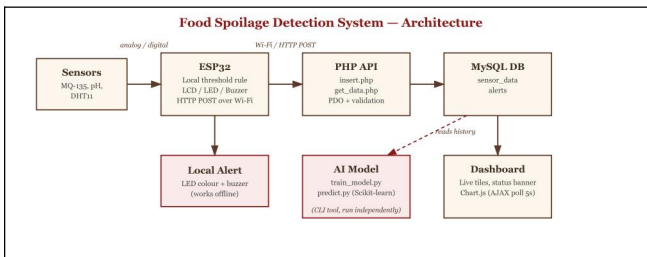


Figure 1: Overall architecture of the IoT-based food spoilage detection system.

3 System Operation

This section walks through one complete monitoring cycle end to end, from sensor sampling to dashboard update, to make the data flow described in Section 2.1 concrete.

3.1 Step-by-Step Workflow

- Sampling: every 5 seconds, the ESP32 reads the MQ-135 gas value, the pH probe's raw ADC value (converted with readPH()), and temperature/humidity from the DHT11.
- Local decision: the firmware applies the threshold rule immediately, without waiting for the server, and updates the LCD, the green/yellow/red status LEDs, and the buzzer (only sounded for Spoiled).
- Transmission: the reading is packaged as an HTTP POST body and sent to api/insert.php over Wi-Fi. If Wi-Fi is down, the ESP32 skips the upload but continues local sensing and alerting.
- Server-side validation and storage: insert.php checks that all four fields are present and numeric, recomputes status using the same

rule, writes a new row to sensor_data, and — if the status is not Fresh — writes a corresponding row to alerts.

- Dashboard refresh: the browser dashboard polls get_data.php every 5 seconds for the latest reading, a rolling history window, and recent alerts, and updates the tiles, status banner, and Chart.js line charts without a page reload.
- Offline intelligence (optional, asynchronous): independent of the live loop, train_model.py can be run periodically against the accumulated sensor_data table to refit the Random Forest classifier; predict.py then uses the newest model.pkl for any subsequent command-line predictions.

3.2 Operating Modes

The system is designed to degrade gracefully rather than fail outright. With hardware connected and Wi-Fi available, it runs in full mode (sensing, local alerting, remote logging, and dashboard visualization). With Wi-Fi unavailable, it runs in offline-local mode (sensing and local alerting only, via the LCD/LEDs/buzzer). Without any hardware attached, the API and dashboard can still be exercised using curl or any HTTP client, which is how the system was developed and tested before the physical sensors were assembled.

3.3 Overall Diagram

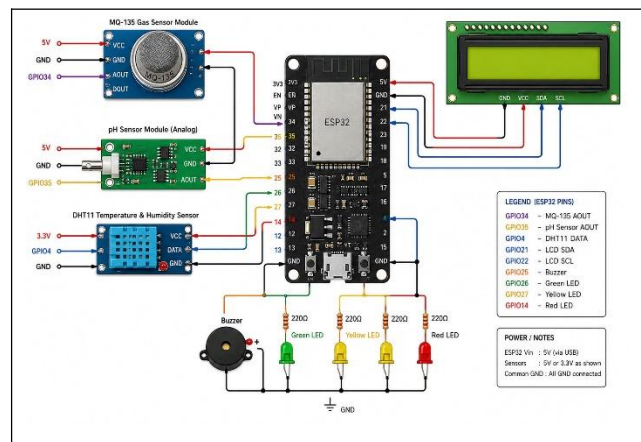


Figure 2. Complete ESP32 wiring diagram showing sensor, display, and output connections for the Food Spoilage Detection System.

4 Results

The complete pipeline was exercised end-to-end using both curl-simulated POST requests and the assembled ESP32 hardware. Readings submitted to insert.php were correctly classified according to the threshold rule and appeared on the dashboard within one polling interval (five seconds), confirming that the sensing, API, storage, and presentation layers interoperate correctly.

The API layer correctly rejected malformed requests: submissions with missing or non-numeric fields returned an HTTP 400 error, and non-POST requests

to insert.php returned HTTP 405, preventing invalid data from entering the database.

With no trained model present, predict.py reproduced the same classification as insert.php for identical inputs, confirming that the rule-based fallback keeps the AI component consistent with the rest of the system. When a model is trained, train_model.py reports precision, recall, and F1-score on a held-out test split, giving a quantitative measure of classification quality on the data collected so far.

Table 1. Representative test cases exercised against the pipeline

Test Case (Input)	Observed Result
Gas 120, pH 6.8 (Fresh case)	Fresh
Gas 300, pH 6.5 (Warning case)	Warning
Gas 450, pH 5.0 (Spoiled case)	Spoiled
Missing 'ph' field	HTTP 400 (rejected)
GET request to insert.php	HTTP 405 (rejected)
predict.py, no model.pkl present	Rule fallback matches API

5 Sample Output

This section shows representative output from each layer of the system for a single reading, to make the data format concrete alongside the workflow described in Section 4.

5.1 LCD Display (ESP32)

For a Warning-level reading, the 1602 LCD cycles through two lines:

```
Gas:300 pH:6.5
Status: WARNING
```

5.2 API Response (get_data.php)

A request to get_data.php?type=latest returns the most recent row as JSON:

```
{
  "id": 42,
  "gas_value": 300,
  "ph_value": 6.5,
  "temperature": 27.8,
  "humidity": 65,
  "status": "Warning",
  "created_at": "2026-08-08 14:32:05"
}
```

5.3 Dashboard View

The live dashboard (dashboard/index.php) shows the same reading as four numeric tiles (gas, pH, temperature, humidity), a colour-coded status banner — green for Fresh, amber for Warning, red for Spoiled — and two auto-updating Chart.js line charts trending gas and pH over the last 30 readings. The alerts panel lists this reading alongside the message logged in the alerts table, most recent first. dashboard/charts.php additionally renders a bar chart of how many readings fall into each status category over the full history.

6 Cost Analysis

All software used in the system — PHP, MySQL, Python, Scikit-learn, Chart.js, and the Arduino/ESP32 toolchain — is free and open-source, and the backend runs locally via Laragon, so the cost of building one monitoring unit is essentially the hardware bill of materials. This significantly reduces the overall development cost and makes the system affordable for educational institutions, research projects, and small-scale deployments. Since no commercial software licenses or cloud hosting services are required, the recurring operational cost is also minimal. The itemized breakdown below is taken directly from the purchase invoice (Bill ID ICE27072026EM, Electronics Mart BD).

Table 2. Bill of materials (Bill ID ICE27072026EM, Electronics Mart BD)

Component	Qty	Unit (BDT)	Subtotal (BDT)
ESP32 – 38 Pin Type-C	1	600.00	600.00
MQ-135 gas sensor	1	180.00	180.00
DHT11 budget temp/humidity sensor	1	175.00	175.00
1602 LCD display	1	170.00	170.00
I ² C module for LCD display	1	105.00	105.00
5V buzzer	2	19.00	38.00
Breadboard, medium (half size)	1	95.00	95.00
Jumper cable, M-M, 40 pcs set	1	110.00	110.00
Jumper cable, M-F, 40 pcs set	1	100.00	100.00
LED 5mm, red	4	2.00	8.00
LED 5mm, yellow	4	2.00	8.00
LED 5mm, green	4	2.00	8.00
220 Ω resistor	5	0.50	2.50

Subtotal: BDT 1,599.50 Discount (2%): –BDT 31.99

Grand total: BDT 1,567.51

The pH sensor module is not part of this invoice — it was sourced separately, as noted in the firmware header comment. A standalone deployment also needs a power source independent of a PC, since the dashboard's PC/Laragon server can stay plugged in but the ESP32 node itself is typically placed near the stored food. These two items are listed separately below.

Table 3. Additional components (procured separately)

Component	Qty	Unit (BDT)	Subtotal (BDT)
pH sensor module (BNC probe)	1	2,500.00	2,500.00
5V/2A USB power adapter	1	250.00	250.00

Invoiced hardware (Table 1): BDT 1,567.51 + Additional components (Table 2): BDT 2,750.00 = **Total project cost: BDT 4,317.51 (≈ US\$36)**

The pH sensor alone accounts for roughly 58% of the total hardware cost, making it the single largest line item in the system — substantially more than the ESP32 itself.

Even so, the total project cost remains well under the price of a single commercial cold-chain data logger, confirming that the system meets its design goal of

low-cost, accessible spoilage monitoring. For larger deployments, the pH module and the second spare buzzer already included as a backup in the invoice are the most straightforward line items to trim, since gas and temperature/humidity alone can already support Warning-level alerting.

7 Industrial and Market Demand

Food spoilage is a large and persistent problem at every stage of the supply chain. The Food and Agriculture Organization of the United Nations estimates that roughly 14% of food produced worldwide is lost between harvest and retail, with a further 17% wasted across households, food service, and retail combined — losses that real-time spoilage monitoring can help reduce by triggering earlier intervention rather than relying on periodic manual checks.

This demand is reflected in market growth for adjacent technologies: the global smart food bin market was valued at roughly US\$201.5 million in 2022 and is projected to reach US\$741.7 million by 2032, while the broader food waste management market is projected to reach approximately US\$54.29 billion by 2028. Gas-sensing approaches to spoilage detection specifically are highlighted in recent review literature as a promising, low-cost route toward commercial, real-time food quality sensors.

Within this landscape, a low-cost ESP32-based system like the one described here is best suited to segments where commercial cold-chain sensor suites are too expensive to justify: individual households, small grocery and fish/meat retailers, campus or restaurant kitchens, and small cold-storage or warehouse operators in markets such as Bangladesh. For these users, a monitoring unit costing roughly BDT 4,318 (Section 5) that gives an immediate visual/audible warning and a historical dashboard offers a meaningful upgrade over manual inspection, even without matching the precision of laboratory-grade or industrial cold-chain equipment. Scaling toward larger commercial deployments would primarily require the multi-node and authentication extensions discussed in Section 7.

8 Future Work

- Recalibrate the pH sensor against pH 4, 7, and 10 buffer solutions and replace the linear ADC approximation with a proper calibration curve.
- Collect a larger, more diverse labelled dataset (target 200+ readings per class) and retrain the Random Forest model, then compare its decision boundaries against the current fixed thresholds.
- Add a device_id field to sensor_data and the API so multiple ESP32 nodes can report into

the same dashboard, supporting multi-room or multi-shelf deployments.

- Replace AJAX polling with WebSockets to reduce dashboard latency and server load as the number of nodes grows.
- Add basic authentication and HTTPS in front of the API before any deployment outside a trusted local network.
- Evaluate a lower-cost pH-free variant (gas + temperature/humidity only) for price-sensitive deployments, as discussed in Section 7.

9 Discussion

The layered design allows each component to be validated independently: the API can be tested without any hardware attached, the dashboard renders correctly from seeded sample data, and the AI component degrades gracefully to rule-based logic when training data is insufficient. This reduces the risk of a single point of failure blocking end-to-end testing during development.

The main limitation is the pH sensor's placeholder calibration, which should be replaced with a buffer-solution calibration before the readings are used quantitatively. The fixed threshold rule is also a simplification of a continuous spoilage process; classification is expected to improve once the Random Forest model is trained on a larger and more diverse labelled dataset. The current design supports a single sensing node and uses polling rather than push-based updates; adding a device identifier field and moving to WebSockets are natural extensions for multi-node or lower-latency deployments. No authentication currently protects the API, which is acceptable on a local network but would need to be addressed before any public-facing deployment.

10 Conclusions

This project demonstrates a working, end-to-end IoT pipeline for food freshness monitoring: sensor acquisition on an ESP32, transmission and persistence through a PHP/MySQL backend, real-time visualization on a web dashboard, and an upgrade path from a fixed rule to a trained machine-learning classifier. The modular architecture keeps each layer independently testable and allows the system to keep functioning, in a degraded but correct form, even when a given layer (network, trained model) is temporarily unavailable.

Acknowledgements

The author thanks the Department of Information and Communication Engineering, Daffodil International University, for the instruction and course framework that guided this simulation project

References

- [1] Espressif Systems. ESP32 Arduino Core Documentation. <https://docs.espressif.com/projects/arduino-esp32/>
- [2] Adafruit Industries. DHT sensor library and Adafruit Unified Sensor documentation. <https://github.com/adafruit/DHT-sensor-library>
- [3] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830, 2011.
- [4] PHP Documentation Group. PDO — PHP Data Objects. <https://www.php.net/manual/en/book.pdo.php>
- [5] Chart.js Contributors. Chart.js Documentation. <https://www.chartjs.org/docs/>
- [6] Food and Agriculture Organization of the United Nations (FAO). *Global Food Losses and Food Waste: Extent, Causes and Prevention*. Rome, 2011.
- [7] Market.us. *Food Waste Statistics By Household, Retail, Solutions*, 2026.
- [8] ResearchAndMarkets.com. *Food Waste Management Global Market to 2028*.
- [9] ScienceDirect. *Advances in Food Spoilage Detection Through Gas Sensing: From Material Insights to Prospects for Commercialisation*, 2026.